

Національний технічний університет України «Київський
політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Дискретна математика

Лабораторна робота №2

«Бінарні відношення та їх основні властивості, операції над
відношеннями»

Виконав:
студент групи ІО-41
Давидчук А. М.
Залікова книжка № 4106

Перевірів
Пономаренко А. М.

Тема: «Бінарні відношення та їх основні властивості, операції над відношеннями».

Мета: вивчити основні властивості бінарних відношень та опанувати операціями над бінарними відношеннями.

Загальне завдання:

Етап 1. Створити програму, яка коректно формує відношення відповідно до варіанту завдання та виконує операції над цими відношеннями.

Етап 2. Ввести елементи множин A та B . Наприклад: A : {Антоніна, Оксана, Галина, Ольга, Світлана, Петро, Тетяна, Іван, Катерина, Олег}, B : {Борис, Василь, Максим, Ольга, Тетяна, Іван, Аркадій, Артем, Оксана, Петро}.

Етап 3. Задати програмно відношення S і R між елементами множин A і B . Наприклад, використовуючи варіант: aSb , якщо a сестра b . aRb , якщо a дружина b .

Теоретичні відомості:

Упорядкована пара – сукупність двох елементів, розташованих у визначеному порядку (a, b) , де порядок елементів має значення;

Бінарне відношення (R) – підмножина декартового добутку множин $X \times Y$, тобто множина впорядкованих пар, що відповідають певній умові;

Декартовий добуток $(X \times Y)$ – множина всіх можливих впорядкованих пар (x, y) , де $x \in X, y \in Y$;

Область визначення $D(R)$ – множина перших елементів усіх пар у відношенні R ;

Область значень $E(R)$ – множина других елементів усіх пар у відношенні R ;

Рефлексивність – відношення R рефлексивне, якщо для кожного $a \in A$ виконується $(a, a) \in R$;

Антирефлексивність – відношення R антирефлексивне, якщо для жодного $a \in A$ не виконується $(a, a) \in R$;

Симетричність – відношення R симетричне, якщо з $(a, b) \in R$ випливає $(b, a) \in R$;

Антисиметричність – відношення R антисиметричне, якщо з $(a, b) \in R$ та $(b, a) \in R$ випливає $a = b$;

Транзитивність – відношення R транзитивне, якщо з $(a, b) \in R$ і $(b, c) \in R$ випливає $(a, c) \in R$;

Еквівалентність – відношення, яке одночасно є рефлексивним, симетричним і транзитивним;

Об'єднання $R_1 \cup R_2$ – множина пар, що належать хоча б одному з відношень R_1 або R_2 ;

Перетин $R_1 \cap R_2$ – множина пар, що належать обом відношенням R_1 і R_2 ;

Різниця $R_1 \setminus R_2$ – множина пар, що належать R_1 , але не належать R_2 ;

Доповнення $U \setminus R$ – множина всіх можливих пар з U , які не належать R ;

Обернене відношення R^{-1} – множина пар, отримана перестановкою елементів у кожній парі (b, a) замість (a, b) ;

Композиція відношень $R \circ S$ – множина пар (a, c) , якщо існує такий b , що $(a, b) \in R$ і $(b, c) \in S$;

Транзитивне замикання R^+ – найменше транзитивне відношення, яке містить R ;

Рефлексивне замикання R^* – найменше рефлексивне відношення, що містить R ;

Список впорядкованих пар – явне перерахування всіх пар, що входять у відношення;

Матриця відношення – квадратна матриця, де $a_{ij} = 1$, якщо існує зв'язок між i -тим і j -тим елементом, і 0 інакше;

Графічне представлення – орієнтований граф, де вершини відповідають елементам множини, а дуги – зв'язкам між ними.

Індивідуальне завдання:

За кодом, мій варіант = 18:

```
>>> G=41
>>> N=6
>>> print("Мій варіант:", (N+G%60)%30+1)
Мій варіант: 18
```

Тобто S – a мати b ; R – a свекруха b :

18	aSb , якщо a мати b .	aRb , якщо a свекруха b .
----	-----------------------------	---------------------------------

Алгоритм формування відношення S (a – мати b):

1. Початок

Маємо дві множини осіб: A та B

Маємо інформацію про стать кожної особи (словник gender)

2. Ітерація по всіх парах (a, b)

Для кожного $a \in A$

Для кожного $b \in B$

Виконуємо логічну перевірку $\text{gender}[a] = \text{'female'}$:

Якщо так, то додаємо пару (a, b) в множину S

Алгоритм формування відношення R (a – свекруха b):

1. Початок

Маємо дві множини осіб: A та B

Маємо інформацію про стать кожної особи (словник gender)

2. Ітерація по всіх парах (a, b)

Для кожного $a \in A$

Для кожного $b \in B$

Виконуємо логічну перевірку $\text{gender}[a] = \text{'female'}$, $\text{gender}[b] = \text{'male'}$:

Якщо так, то додаємо пару (a, b) в множину R

Код:

Файл *main.py*:

```
import tkinter as tk
from tkinter import messagebox, filedialog
import logic

# Попередньо визначені списки імен для вибору
female_names = ["Антоніна", "Оксана", "Галина", "Ольга", "Світлана", "Тетяна",
                "Катерина"]
male_names    = ["Петро", "Іван", "Олег", "Борис", "Василь", "Максим", "Аркадій",
                "Артем"]

# Глобальні множини A і B (спочатку порожні)
set_A = []
set_B = []

# Словник, що визначає статі осіб
gender = {
    "Антоніна": "female", "Оксана": "female", "Галина": "female", "Ольга":
    "female",
    "Світлана": "female", "Тетяна": "female", "Катерина": "female",
    "Петро": "male", "Іван": "male", "Олег": "male", "Борис": "male",
    "Василь": "male", "Максим": "male", "Аркадій": "male", "Артем": "male"
}

# Допоміжна функція для побудови рядкового представлення матриці
def matrix_to_string(matrix, row_labels, col_labels):
    """
    Формує відформатований рядок для виводу матриці.
    row_labels – підписи рядків (елементи множини A),
    col_labels – підписи стовпців (елементи множини B).
    matrix[i][j] – 0 або 1, залежно від того, належить чи ні пара (A[i], B[j])
    відношенню.
    """

    # Ширина кожної колонки (можна змінити за потреби)
    col_width = 10
```

```

# Формуємо заголовок таблиці
# Спочатку порожній відступ для кута таблиці
header = " " * col_width
# Додаємо назви стовпців (елементів B)
for col in col_labels:
    header += f"{col:>{col_width}}"
header += "\n"

# Формуємо самі рядки матриці
rows_str = ""
for i, row in enumerate(matrix):
    # Вирівнюємо назву рядка (елемент A) ліворуч, решту – праворуч
    row_str = f"{row_labels[i]:<{col_width}}"
    for val in row:
        row_str += f"{val:>{col_width}}"
    rows_str += row_str + "\n"

return header + rows_str

# Вікно 1: Головне меню
class MainWindow(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Лабораторна робота. Головне меню")
        self.geometry("400x300")

        # Інформація про студента, групу та варіант
        self.group_number = 41 # приклад значення
        self.student_index = 6 # приклад значення
        self.variant = logic.calculate_variant(self.group_number,
self.student_index)
        info_text = (f"Студент: Давидчук Артем\n"
                    f"Група: ІО - {self.group_number}\n"
                    f"Номер у групі: {self.student_index}\n"
                    f"Варіант: {self.variant}")
        tk.Label(self, text=info_text, justify="left").pack(pady=10)

        # Кнопки для відкриття інших вікон
        btn_frame = tk.Frame(self)
        btn_frame.pack(pady=20)
        tk.Button(btn_frame, text="Вікно 2: Робота з множинами",
command=self.open_window2).grid(row=0, column=0, padx=5, pady=5)
        tk.Button(btn_frame, text="Вікно 3: Відношення S та R",
command=self.open_window3).grid(row=1, column=0, padx=5, pady=5)
        tk.Button(btn_frame, text="Вікно 4: Операції над відношеннями",
command=self.open_window4).grid(row=2, column=0, padx=5, pady=5)

    def open_window2(self):
        win2 = Window2(self)
        win2.grab_set()

    def open_window3(self):
        win3 = Window3(self)
        win3.grab_set()

    def open_window4(self):
        win4 = Window4(self)
        win4.grab_set()

# Вікно 2: Робота з множинами A і B
class Window2(tk.Toplevel):
    def __init__(self, master):

```

```

super().__init__(master)
self.title("Вікно 2: Робота з множинами")
self.geometry("700x400")
self.resizable(False, False)

# Radiobutton для вибору, до якої множини додавати імена
self.target_set = tk.StringVar(value="A")
frame_radio = tk.Frame(self)
frame_radio.pack(pady=5)
tk.Label(frame_radio, text="Виберіть, до якої множини додавати імена:").pack(side="left")
tk.Radiobutton(frame_radio, text="Множина A", variable=self.target_set, value="A").pack(side="left")
tk.Radiobutton(frame_radio, text="Множина B", variable=self.target_set, value="B").pack(side="left")

# Listbox для жіночих імен
frame_female = tk.Frame(self)
frame_female.pack(side="left", padx=10, pady=10, fill="y")
tk.Label(frame_female, text="Жіночі імена").pack()
self.listbox_female = tk.Listbox(frame_female, selectmode=tk.MULTIPLE, exportselection=False)
for name in female_names:
    self.listbox_female.insert(tk.END, name)
self.listbox_female.pack()
tk.Button(frame_female, text="Додати вибрані", command=self.add_selected_names).pack(pady=5)

# Listbox для чоловічих імен
frame_male = tk.Frame(self)
frame_male.pack(side="left", padx=10, pady=10, fill="y")
tk.Label(frame_male, text="Чоловічі імена").pack()
self.listbox_male = tk.Listbox(frame_male, selectmode=tk.MULTIPLE, exportselection=False)
for name in male_names:
    self.listbox_male.insert(tk.END, name)
self.listbox_male.pack()
tk.Button(frame_male, text="Додати вибрані", command=self.add_selected_names).pack(pady=5)

# Кнопки для роботи з файлами та очищення множин
frame_files = tk.Frame(self)
frame_files.pack(pady=10)
tk.Button(frame_files, text="Зберегти множину A", command=self.save_set_A).grid(row=0, column=0, padx=5)
tk.Button(frame_files, text="Зчитати множину A", command=self.load_set_A).grid(row=0, column=1, padx=5)
tk.Button(frame_files, text="Очистити множину A", command=self.clear_set_A).grid(row=0, column=2, padx=5)
tk.Button(frame_files, text="Зберегти множину B", command=self.save_set_B).grid(row=1, column=0, padx=5)
tk.Button(frame_files, text="Зчитати множину B", command=self.load_set_B).grid(row=1, column=1, padx=5)
tk.Button(frame_files, text="Очистити множину B", command=self.clear_set_B).grid(row=1, column=2, padx=5)

def add_selected_names(self):
    target = self.target_set.get()
    selected_female = [self.listbox_female.get(i) for i in self.listbox_female.curselection()]
    selected_male = [self.listbox_male.get(i) for i in self.listbox_male.curselection()]

```

```

names = selected_female + selected_male
global set_A, set_B
if target == "A":
    for name in names:
        if name not in set_A:
            set_A.append(name)
    messagebox.showinfo("Інформація", "Імена додано до множини A")
else:
    for name in names:
        if name not in set_B:
            set_B.append(name)
    messagebox.showinfo("Інформація", "Імена додано до множини B")

def save_set_A(self):
    global set_A
    filename = filedialog.asksaveasfilename(defaultextension=".txt",
title="Зберегти множину A")
    if filename:
        with open(filename, "w", encoding="utf-8") as f:
            for item in set_A:
                f.write(item + "\n")
        messagebox.showinfo("Інформація", "Множину A збережено.")

def load_set_A(self):
    global set_A
    filename = filedialog.askopenfilename(title="Зчитати множину A",
filetypes=[("Text files", "*.txt")])
    if filename:
        with open(filename, "r", encoding="utf-8") as f:
            set_A = [line.strip() for line in f if line.strip()]
        messagebox.showinfo("Інформація", "Множину A зчитано.")

def clear_set_A(self):
    global set_A
    set_A = []
    messagebox.showinfo("Інформація", "Множину A очищено.")

def save_set_B(self):
    global set_B
    filename = filedialog.asksaveasfilename(defaultextension=".txt",
title="Зберегти множину B")
    if filename:
        with open(filename, "w", encoding="utf-8") as f:
            for item in set_B:
                f.write(item + "\n")
        messagebox.showinfo("Інформація", "Множину B збережено.")

def load_set_B(self):
    global set_B
    filename = filedialog.askopenfilename(title="Зчитати множину B",
filetypes=[("Text files", "*.txt")])
    if filename:
        with open(filename, "r", encoding="utf-8") as f:
            set_B = [line.strip() for line in f if line.strip()]
        messagebox.showinfo("Інформація", "Множину B зчитано.")

def clear_set_B(self):
    global set_B
    set_B = []
    messagebox.showinfo("Інформація", "Множину B очищено.")

```

Вікно 3: Відображення множин і матричних представлень відношень S та R

```

class Window3(tk.Toplevel):
    def __init__(self, master):
        super().__init__(master)
        self.title("Вікно 3: Відношення S та R")
        # Збільшуємо розміри вікна, щоб умістити більші текстові поля:
        self.geometry("900x600")

        # Відображення множин A та B
        frame_sets = tk.Frame(self)
        frame_sets.pack(pady=10)
        tk.Label(frame_sets, text="Множина A: " + " ".join(set_A)).pack()
        tk.Label(frame_sets, text="Множина B: " + " ".join(set_B)).pack()

        # Обчислення відношень S та R
        self.relation_S = logic.build_relation_S(set_A, set_B, gender)
        self.relation_R = logic.build_relation_R(set_A, set_B, gender)

        # -----
        # Відображення матриці відношення S
        # -----
        tk.Label(self, text="Матриця відношення S (а мати b):").pack()

        # Фрейм для текстового поля S + скролбари
        frame_S = tk.Frame(self)
        frame_S.pack(pady=5, fill="both", expand=True)

        # Скролбари (вертикальний і горизонтальний)
        scrollbar_y_S = tk.Scrollbar(frame_S, orient="vertical")
        scrollbar_y_S.pack(side="right", fill="y")

        scrollbar_x_S = tk.Scrollbar(frame_S, orient="horizontal")
        scrollbar_x_S.pack(side="bottom", fill="x")

        matrix_S = logic.build_matrix(self.relation_S, set_A, set_B)
        text_S = tk.Text(
            frame_S,
            height=20,
            width=80,
            wrap="none", # Вимикаємо перенесення слів, щоб горизонтальний скрол
            yscrollcommand=scrollbar_y_S.set,
            xscrollcommand=scrollbar_x_S.set
        )
        text_S.insert(tk.END, matrix_to_string(matrix_S, set_A, set_B))
        text_S.config(state=tk.DISABLED)
        text_S.pack(side="left", fill="both", expand=True)

        # Прив'язуємо скролбари до текстового поля
        scrollbar_y_S.config(command=text_S.yview)
        scrollbar_x_S.config(command=text_S.xview)

        # -----
        # Відображення матриці відношення R
        # -----
        tk.Label(self, text="Матриця відношення R (а свекруха b):").pack()

        # Фрейм для текстового поля R + скролбари
        frame_R = tk.Frame(self)
        frame_R.pack(pady=5, fill="both", expand=True)

        scrollbar_y_R = tk.Scrollbar(frame_R, orient="vertical")
        scrollbar_y_R.pack(side="right", fill="y")

```

```

scrollbar_x_R = tk.Scrollbar(frame_R, orient="horizontal")
scrollbar_x_R.pack(side="bottom", fill="x")

matrix_R = logic.build_matrix(self.relation_R, set_A, set_B)
text_R = tk.Text(
    frame_R,
    height=20,
    width=80,
    wrap="none",
    yscrollcommand=scrollbar_y_R.set,
    xscrollcommand=scrollbar_x_R.set
)
text_R.insert(tk.END, matrix_to_string(matrix_R, set_A, set_B))
text_R.config(state=tk.DISABLED)
text_R.pack(side="left", fill="both", expand=True)

scrollbar_y_R.config(command=text_R.yview)
scrollbar_x_R.config(command=text_R.xview)

# Вікно 4: Відображення операцій над відношеннями
class Window4(tk.Toplevel):
    def __init__(self, master):
        super().__init__(master)
        self.title("Вікно 4: Операції над відношеннями")
        self.geometry("600x600")

        # Оновлення відношень з поточними множинами
        relation_S = logic.build_relation_S(set_A, set_B, gender)
        relation_R = logic.build_relation_R(set_A, set_B, gender)
        U = {(a, b) for a in set_A for b in set_B}

        union_SR = relation_S.union(relation_R)
        intersection_SR = relation_S.intersection(relation_R)
        difference_R_S = relation_R.difference(relation_S)
        difference_U_R = U.difference(relation_R)
        inverse_S = logic.inverse_relation(relation_S)

        # Побудова рядкового представлення операцій
        ops_str = ""
        ops_str += "Матриця  $R \cup S$ :\n" +
matrix_to_string(logic.build_matrix(union_SR, set_A, set_B), set_A, set_B) +
"\n"
        ops_str += "Матриця  $R \cap S$ :\n" +
matrix_to_string(logic.build_matrix(intersection_SR, set_A, set_B), set_A,
set_B) + "\n"
        ops_str += "Матриця  $R \setminus S$ :\n" +
matrix_to_string(logic.build_matrix(difference_R_S, set_A, set_B), set_A, set_B)
+ "\n"
        ops_str += "Матриця  $U \setminus R$ :\n" +
matrix_to_string(logic.build_matrix(difference_U_R, set_A, set_B), set_A, set_B)
+ "\n"
        ops_str += "Матриця  $S^{-1}$ :\n" +
matrix_to_string(logic.build_matrix(inverse_S, set_B, set_A), set_B, set_A) +
"\n"

        text_ops = tk.Text(self, height=30, width=70)
        text_ops.insert(tk.END, ops_str)
        text_ops.config(state=tk.DISABLED)
        text_ops.pack(pady=10)

root = MainWindow()

```

```
root.mainloop()
```

Файл logic.py:

```
def calculate_variant(group_number, student_index):
    """
    Обчислення варіанту за формулою:
     $Z = (student\_index + group\_number \% 60) \% 30 + 1$ 
    """

    return (student_index + group_number % 60) % 30 + 1

def build_relation_S(set_A, set_B, gender):
    """
    Формує відношення S: aSb, якщо a мати b.
    Умови:
    - a повинна бути жіночої статі (gender[a] == 'female')
    - a не може дорівнювати b
    """

    relation_S = set()
    for a in set_A:
        if gender.get(a, '') == 'female':
            for b in set_B:
                if a != b:
                    relation_S.add((a, b))
    return relation_S

def build_relation_R(set_A, set_B, gender):
    """
    Формує відношення R: aRb, якщо a свекруха b.
    Умови:
    - a повинна бути жіночої статі
    - b повинна бути чоловічої статі
    - a не може дорівнювати b
    """

    relation_R = set()
    for a in set_A:
        if gender.get(a, '') == 'female':
            for b in set_B:
                if a != b and gender.get(b, '') == 'male':
                    relation_R.add((a, b))
    return relation_R

def build_matrix(relation, set_A, set_B):
    """
    Створює матричне представлення відношення.
    Рядки відповідають елементам з set_A, а стовпці – з set_B.
    Якщо пара (a, b) належить відношенню, ставимо 1, інакше 0.
    """

    matrix = []
    for a in set_A:
        row = []
        for b in set_B:
            row.append(1 if (a, b) in relation else 0)
        matrix.append(row)
    return matrix

def inverse_relation(relation):
```

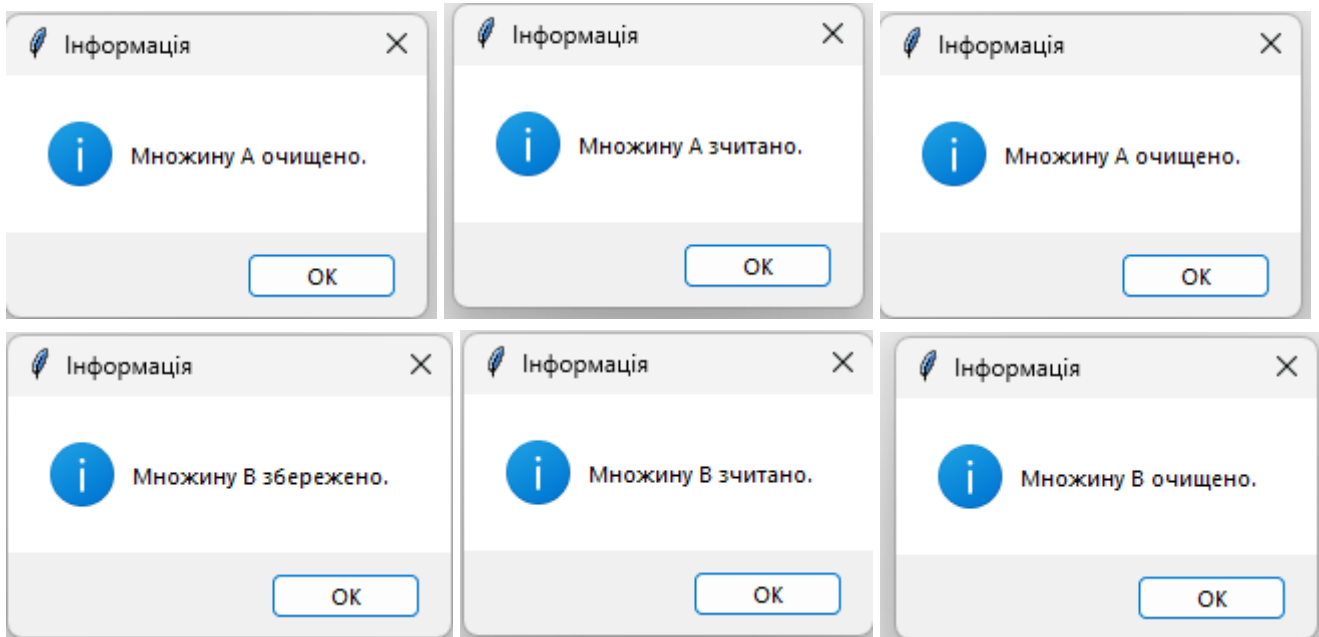
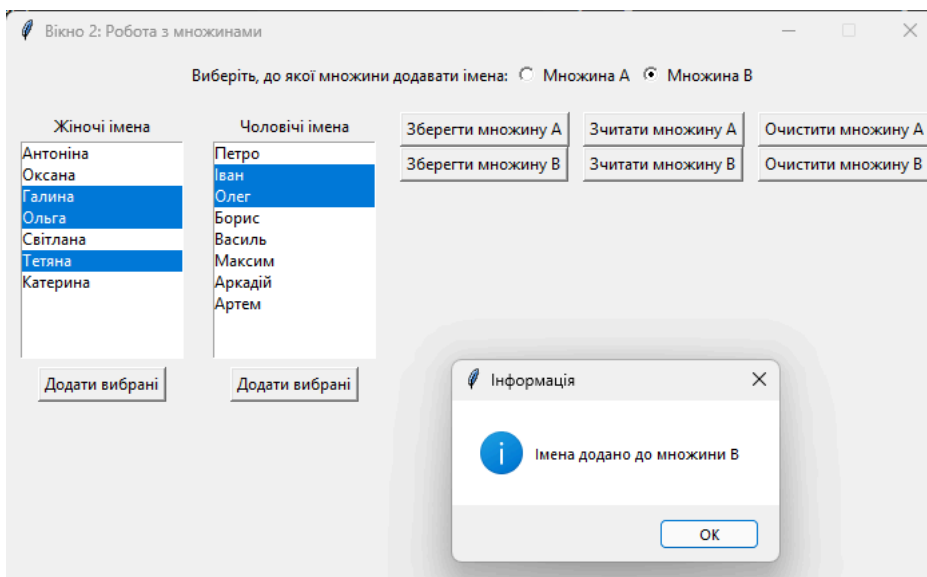
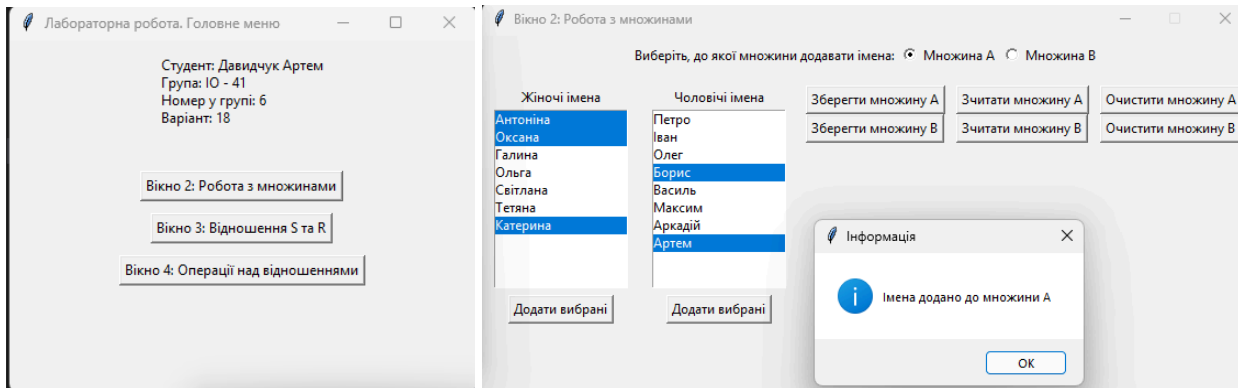
"""

Обчислює обернений зв'язок (інверсію) відношення.

"""

```
return {(b, a) for (a, b) in relation}
```

Результати виконання (знімки екрана):



Вікно 3: Відношення S та R

Множина A: Антоніна, Оксана, Катерина, Борис, Артем

Множина B: Галина, Ольга, Тетяна, Іван, Олег

Матриця відношення S (а мати b):

	Галина	Ольга	Тетяна	Іван	Олег
Антоніна	1	1	1	1	1
Оксана	1	1	1	1	1
Катерина	1	1	1	1	1
Борис	0	0	0	0	0
Артем	0	0	0	0	0

Матриця відношення R (а свекруха b):

	Галина	Ольга	Тетяна	Іван	Олег
Антоніна	0	0	0	1	1
Оксана	0	0	0	1	1
Катерина	0	0	0	1	1
Борис	0	0	0	0	0
Артем	0	0	0	0	0

Вікно 4: Операції над відношеннями

Матриця $R \cup S$:

	Галина	Ольга	Тетяна	Іван	Олег
Антоніна	1	1	1	1	1
Оксана	1	1	1	1	1
Катерина	1	1	1	1	1
Борис	0	0	0	0	0
Артем	0	0	0	0	0

Матриця $R \cap S$:

	Галина	Ольга	Тетяна	Іван	Олег
Антоніна	0	0	0	1	1
Оксана	0	0	0	1	1
Катерина	0	0	0	1	1
Борис	0	0	0	0	0
Артем	0	0	0	0	0

Матриця $R \setminus S$:

	Галина	Ольга	Тетяна	Іван	Олег
Антоніна	0	0	0	0	0
Оксана	0	0	0	0	0
Катерина	0	0	0	0	0
Борис	0	0	0	0	0
Артем	0	0	0	0	0

Матриця $U \setminus R$:

	Галина	Ольга	Тетяна	Іван	Олег
Антоніна	1	1	1	0	0
Оксана	1	1	1	0	0
Катерина	1	1	1	0	0
Борис	1	1	1	1	1

Висновок:

У процесі виконання лабораторної роботи було досліджено теоретичні основи бінарних відношень, реалізовано алгоритм формування відношень S (a мати b) та R (a свекруха b), також було підтверджено у відсутності суперечностей між цими відношеннями. Опановано операції над бінарними відношеннями та їхнє представлення у вигляді матриць. Результати виконання програми продемонстровано на знімках екрана.